

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА**

**ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №2
« Структури даних та функціональна декомпозиція »
з курсу «Професійний Python»
Варіант №5 — « Аналіз каталогу автомобілів»**

Виконала: Хілінська Іванна

Група: ФєП-32

Перевірів: Куньо Іван Михайлович

Дата: 22.09.2026

Львів — 2026

1. Мета роботи

Сформуванати практичні навички професійного використання базових та розширених структур даних Python, навчитися обирати структуру відповідно до задачі, оцінювати складність основних операцій та декомпонувати програму на невеликі функції з чітко визначеною відповідальністю.

У ході роботи необхідно опанувати: list, tuple, set, dict, модуль collections, comprehensions, генераторні вирази, оцінювання часової складності операцій, функціональну декомпозицію, positional та keyword аргументи, *args, kwargs, lambda, closures та decorators.

2. Індивідуальний варіант і постановка задачі

Варіант №5 — «Аналіз каталогу автомобілів». Для кожного автомобіля зберігати марку (make), модель (model), рік випуску (year), ціну (price) та пробіг (mileage). Реалізувати визначення унікальних марок, кількість автомобілів кожної марки, середню ціну, найдорожчий автомобіль, автомобіль із найменшим пробігом, сортування за ціною, фільтрацію за роком випуску та dict-index за ID.

Проект повинен мати рекомендовану структуру (data.py, decorators.py, processors.py, analytics.py, main.py), використовувати list, tuple, set, dict, модуль collections (Counter, defaultdict), list/set/dict comprehensions, щонайменше 6 функцій, *args, kwargs, lambda, closure, decorator, а також оцінку складності та бенчмарк для порівняння пошуку в list та dict.

3. Структура проєкту

```
car_catalog/  
├─ pyproject.toml  
├─ README.md  
├─ .gitignore  
├─ report/  
│   └─ Прізвище_Ім'я_Лаб02_V5.pdf  
├─ tests/  
│   └─ test_services.py  
└─ src/  
    └─ car_catalog/  
        ├─ init.py  
        ├─ main.py  
        ├─ data.py  
        └─ processors.py
```

```
└─ analytics.py
└─ decorators.py
```

Каталог `src/car_catalog` містить пакет застосунку. Файл `data.py` містить початковий набір структурованих даних каталогів автомобілів (списки словників та кортежів). Файл `processors.py` містить функції обробки даних (індексування, групування через `defaultdict`, підрахунок через `Counter` та фільтрації). Файл `analytics.py` містить бізнес-логіку аналізу (обчислення середніх значень, пошук найдорожчого авто, сортування та замикання). Файл `decorators.py` містить декоратор вимірювання часу виконання функцій. Файл `main.py` є точкою входу в консольний застосунок та запускає бенчмарк. `init.py` ініціалізує пакет, а `tests/test_services.py` містить автоматизовані модульні тести для перевірки функцій бізнес-логіки.

4. Опис створених модулів

4.1. data.py

У модулі збережено початкові дані та конфігурацію системи. Конфігурація `CATALOG_METADATA` представлена незмінюваним кортежем `tuple`, а початковий каталог `INITIAL_CARS` реалізовано у вигляді списку `list` з об'єктами `Car`.

```
CATALOG_METADATA: tuple[str, str, int] = ("Car Catalog System", "v2.0", 2026)

INITIAL_CARS: list[Car] = [

    Car("Toyota", "Camry", 2019, 21000.0, 75000),

    Car("BMW", "X5", 2021, 55000.0, 30000),

    Car("Toyota", "RAV4", 2022, 32000.0, 15000),

    Car("Audi", "A6", 2018, 23000.0, 110000),

    Car("Mercedes", "C-Class", 2020, 41000.0, 45000),

]
```

4.2. decorators.py

Модуль містить декоратор вищого порядку `measure_time`, який обгортає функції та обчислює час їх виконання за допомогою `time.perf_counter()`, не змінюючи основну логіку.

```
def measure_time(func: Callable[..., Any]) -> Callable[..., Any]:

    """Decorator to measure execution time of a function."""

    @wraps(func)

    def wrapper(*args: Any, **kwargs: Any) -> Any:

        start = perf_counter()

        result = func(*args, **kwargs)
```

```

        elapsed = perf_counter() - start

        print(f"[BENCHMARK] {func.__name__} executed in: {elapsed:.8f} sec")

        return result

    return wrapper

```

4.3. processors.py

У модулі реалізовано функції обробки та структурування даних. Функція `get_unique_makes` використовує Set Comprehension для створення множини set марок авто, `create_car_index` створює словниковий індекс за допомогою Dict Comprehension, `group_cars_by_make` здійснює групування через `defaultdict(list)`, а `count_cars_by_make` виконує підрахунок кількості за допомогою Counter.

```

def get_unique_makes(cars: list[Car]) -> set[str]:
    """Set Comprehension for unique car makes."""
    return {car.make for car in cars}

def create_car_index(cars: list[Car]) -> dict[int, Car]:
    """Dict Comprehension for indexing by ID."""
    return {i + 1: car for i, car in enumerate(cars)}

def group_cars_by_make(cars: list[Car]) -> dict[str, list[Car]]:
    """defaultdict usage from collections module."""
    grouped = defaultdict(list)
    for car in cars:
        grouped[car.make].append(car)
    return dict(grouped)

def count_cars_by_make(cars: list[Car]) -> Counter:
    """Counter usage from collections module."""
    return Counter(car.make for car in cars)

```

4.4. analytics.py

Модуль містить аналітичні функції та бізнес-логіку. Для обчислення середньої ціни `calculate_average_price` використано декоратор `@measure_time` та генераторний вираз. Функції `find_most_expensive_car`, `find_lowest_mileage_car` та `sort_cars_by_price` використовують lambda-вирази. Для демонстрації синтаксичних можливостей реалізовано

передачу *args у calculate_average_prices_custom, kwargs у create_car_record, а також замикання (Closure) у фабриці фільтрів create_year_filter.

```
@measure_time

def calculate_average_price(cars: list[Car]) -> float:
    """Calculate average price using a generator expression."""
    if not cars:
        return 0.0
    return sum(car.price for car in cars) / len(cars)

def find_most_expensive_car(cars: list[Car]) -> Car | None:
    """Lambda usage for finding the most expensive car."""
    return max(cars, key=lambda c: c.price, default=None)

def find_lowest_mileage_car(cars: list[Car]) -> Car | None:
    """Lambda usage for finding the car with the lowest mileage."""
    return min(cars, key=lambda c: c.mileage, default=None)

def sort_cars_by_price(cars: list[Car], reverse: bool = True) -> list[Car]:
    """Sort cars by price."""
    return sorted(cars, key=lambda c: c.price, reverse=reverse)

def calculate_average_prices_custom(*prices: float) -> float:
    """Calculate average price using positional arguments (*args)."""
    if not prices:
        return 0.0
    return sum(prices) / len(prices)

def create_car_record(**kwargs) -> dict:
    """Create a car dictionary using keyword arguments (**kwargs)."""
    return dict(kwargs)

def create_year_filter(min_year: int) -> Callable[[Car], bool]:
    """Closure factory for filtering cars by minimum year."""
    def predicate(car: Car) -> bool:
```

```
        return car.year >= min_year

    return predicate
```

4.5. main.py

Головний модуль об'єднує роботу всіх компонентів, виводить результати обробки даних у консоль та містить функцію `run_benchmark` для експериментального порівняння часу лінійного пошуку в `list` $O(n)$ та пошуку в `dict` $O(1)$ на вибірках 1 000, 10 000 та 100 000 елементів.

```
from car_catalog.models import Car

from car_catalog.data import INITIAL_CARS, CATALOG_METADATA

from car_catalog.processors import (
    get_unique_makes,
    create_car_index,
    group_cars_by_make,
    count_cars_by_make,
)

from car_catalog.analytics import (
    calculate_average_price,
    find_most_expensive_car,
    find_lowest_mileage_car,
    sort_cars_by_price,
    calculate_average_prices_custom,
    create_car_record,
    create_year_filter,
)

from time import perf_counter

def run_benchmark() -> None:
    """Benchmark comparing List search O(n) vs Dict lookup O(1)."""

    print("\n" + "-" * 65)

    print("EXPERIMENTAL PART: BENCHMARK")

    print("-" * 65)

    sizes = [1000, 10000, 100000]

    print(f"{'Record Count':<18} | {'List Search (sec)':<20} | {'Dict Search (sec)':<20}")
```

```

print("-" * 65)

for n in sizes:

    test_cars = [Car("Make", f"Model_{i}", 2020, 10000.0, 50000) for i in range(n)]

    target_id = n - 1

    start = perf_counter()

    _ = next((c for c in test_cars if c.model == f"Model_{target_id}"), None)

    list_time = perf_counter() - start

    car_dict = {f"Model_{i}": c for i, c in enumerate(test_cars)}

    start = perf_counter()

    _ = car_dict.get(f"Model_{target_id}")

    dict_time = perf_counter() - start

    print(f"{n:<18} | {list_time:<20.8f} | {dict_time:<20.8f}")

def main() -> None:

    print(f"--- {CATALOG_METADATA[0]} ({CATALOG_METADATA[1]}) ---")

    cars = INITIAL_CARS.copy()

    print("\n1. All Cars:")

    for car in cars:

        print(f" - {car.full_title}: ${car.price:.2f}, {car.mileage} km")

    print("\n2. Unique Makes (set):", get_unique_makes(cars))

    avg_price = calculate_average_price(cars)

    print(f"\n3. Average Price: ${avg_price:.2f}")

    best_car = find_most_expensive_car(cars)

    if best_car:

        print(f" Most expensive car: {best_car.full_title} (${best_car.price:.2f})")

```

```

min_mileage_car = find_lowest_mileage_car(cars)

if min_mileage_car:
    print(f"    Lowest mileage car: {min_mileage_car.full_title} ({min_mileage_car.mileage}
km)")

print("\n4. Sorted by price (descending):")
for car in sort_cars_by_price(cars):
    print(f"    - {car.full_title}: ${car.price:.2f}")

print("\n5. Car count by make (Counter):", dict(count_cars_by_make(cars)))

print("    Grouped cars by make (defaultdict):")
for make, group in group_cars_by_make(cars).items():
    print(f"        * {make}: {len(group)} items")

index = create_car_index(cars)

print("\n6. Search by ID (Dict Index) [ID=2]:", index.get(2))

filter_2020_plus = create_year_filter(2020)
newer_cars = [c for c in cars if filter_2020_plus(c)]

print(f"\n7. Cars from 2020+ (Closure & List Comprehension): {len(newer_cars)} items")

print("\n8. Demo *args:", calculate_average_prices_custom(10000.0, 20000.0, 30000.0))

print("    Demo **kwargs:", create_car_record(make="Tesla", model="Model 3", year=2023,
price=45000.0))

run_benchmark()

if __name__ == "__main__":
    main()

```

5. Приклад результату

Фактичний результат запуску згенерованого проєкту:

--- Car Catalog System (v2.0) ---

1. All Cars:

- Toyota Camry (2019): \$21000.00, 75000 km
- BMW X5 (2021): \$55000.00, 30000 km
- Toyota RAV4 (2022): \$32000.00, 15000 km
- Audi A6 (2018): \$23000.00, 110000 km
- Mercedes C-Class (2020): \$41000.00, 45000 km

2. Unique Makes (set): {'BMW', 'Audi', 'Mercedes', 'Toyota'}

[BENCHMARK] calculate_average_price executed in: 0.00000830 sec

3. Average Price: \$34400.00

Most expensive car: BMW X5 (2021) (\$55000.00)

Lowest mileage car: Toyota RAV4 (2022) (15000 km)

4. Sorted by price (descending):

- BMW X5 (2021): \$55000.00
- Mercedes C-Class (2020): \$41000.00
- Toyota RAV4 (2022): \$32000.00
- Audi A6 (2018): \$23000.00
- Toyota Camry (2019): \$21000.00

5. Car count by make (Counter): {'Toyota': 2, 'BMW': 1, 'Audi': 1, 'Mercedes': 1}

Grouped cars by make (defaultdict):

- * Toyota: 2 items
- * BMW: 1 items
- * Audi: 1 items
- * Mercedes: 1 items

6. Search by ID (Dict Index) [ID=2]: Car(make='BMW', model='X5', year=2021, price=55000.0, mileage=30000)

7. Cars from 2020+ (Closure & List Comprehension): 3 items

8. Demo *args: 20000.0

Demo **kwargs: {'make': 'Tesla', 'model': 'Model 3', 'year': 2023, 'price': 45000.0}

EXPERIMENTAL PART: BENCHMARK

Record Count	List Search (sec)	Dict Search (sec)
1000	0.00014330	0.00000150
10000	0.00250630	0.00000470
100000	0.01412310	0.00000480

6. Аналіз складності

Оцінювання теоретичної алгоритмічної складності основних операцій обробки даних у каталозі автомобілів.

Пошук у list

Перебір elements у списку cars для знаходження об'єкта за модельною назвою. В найгіршому та середньому випадках складність становить $O(n)$.

```
_ = next((c for c in test_cars if c.model == f"Model_{target_id}"), None)
```

Побудова та пошук у dict index

Формування словника за допомогою dict comprehension має складність $O(n)$. Проте наступні операції точкового пошуку за ключем виконуються за $O(1)$ у середньому завдяки механізму хешування.

```
car_dict.get(f"Model_{target_id}")
```

Перевірка унікальності та наявності елемента у set

Формування set за допомогою set comprehension вимагає $O(n)$. Перевірка наявності елемента ("Toyota" in makes) виконується за $O(1)$ у середньому.

```
return {car.make for car in cars}
```

Сортування за ціною

Сортування списку об'єктів виконується за допомогою вбудованого алгоритму Timsort, який має часову складність $O(n \log n)$.

```
return sorted(cars, key=lambda c: c.price, reverse=reverse)
```

Агрегація та обчислення середнього

Обчислення суми елементів за допомогою `sum()` із генераторним виразом вимагає одноразового проходу по масиву, що відповідає складності $O(n)$.

```
sum(car.price for car in cars) / len(cars)
```

7. Робота з Git

У демонстраційному проєкті створено три логічні commits.

```
git add src/car_catalog/data.py
```

```
git commit -m "Setup data"
```

```
git add src/car_catalog/processors.py src/car_catalog/analytics.py src/car_catalog/decorators.py
```

```
git commit -m "Implement data processors, analytics functions and time decorato"
```

```
git add src/car_catalog/main.py
```

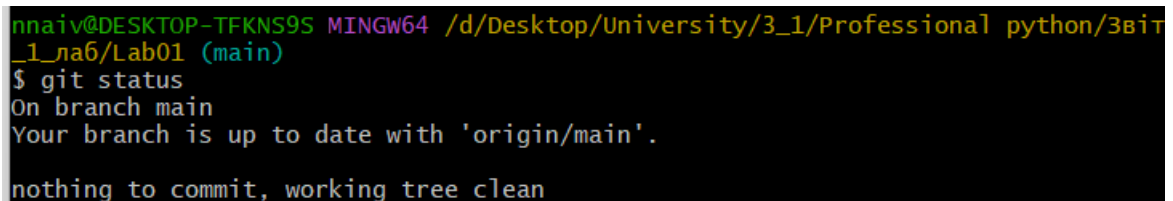
```
git commit -m "Add main for output"
```

7.1 Результат git status

On branch main

Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean



```
nnaiv@DESKTOP-TFKNS9S MINGW64 /d/Desktop/University/3_1/Professional python/3vit
_1_лаб/Lab01 (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
```

7.2. Результат git log --oneline

```
01da0be (HEAD -> main, origin/main) Add main for output
```

```
3946a88 Implement data processors, analytics functions and time decorato
```

```
9f1ce1e Setup data
```

```
nnatv@DESKTOP-IFKNS9S MINGW64 /d/Desktop/University/3_1/Professional python
_1_лаб/Lab01 (main)
$ git log --oneline
01da0be (HEAD -> main, origin/main) Add main for output
3946a88 Implement data processors, analytics functions and time decorato
9f1ce1e Setup data
```

8. Висновок

У лабораторній роботі №2 відбувається перехід від базового застосування колекцій Python до свідомого добору оптимальних структур даних і конструювання модульної архітектури для обробки інформації. Застосування списків, кортежів, множин, словників та спеціалізованих модулів із collections забезпечує гнучке відображення різноманітних об'єктів відповідно до їхніх внутрішніх властивостей. Компактне створення нових контейнерів досягається за допомогою генераторних виразів, а аналіз за системою Big-O дозволяє об'єктивно прогнозувати масштабованість розробленого алгоритму. Одночасно грамотна декомпозиція, використання довільних аргументів, лямбда-функцій, замикань і декораторів створюють надійну базу для подальшого опанування професійної розробки, об'єктно-орієнтованого підходу, автотестування та проєктування гнучких програмних систем. Підсумком виконаної роботи стає автономний модуль обробки структурованої інформації, готовий до інтеграції як повноцінний компонент у наскрізний навчальний проєкт.